

Hermes + Feishu/Lark Research Agent 实现 Schema

0. 项目名称

arxiv-feishu-research-agent

目标是实现一个可长期运行的飞书研究助理：

1. 每天新加坡时间 08:30 自动汇总过去 24 小时内 arXiv 新发表、且与用户研究方向匹配的论文，生成中文晨报并发送到指定飞书群。
2. 用户可以在飞书群里 @ 机器人，让它执行即时调研命令，例如查新论文、总结论文、判断是否与用户当前研究撞车、把论文保存到飞书文档或多维表格。
3. 系统必须支持语义匹配、arXiv 分类匹配、关键词匹配、去重、相关性评分、摘要、反馈学习和审计日志。
4. 系统必须默认安全，不允许 Agent 任意调用 shell 或任意修改飞书资源。

1. 事实基线与兼容性约束

Hermes Agent 的 Feishu/Lark 集成文档说明，它可以作为飞书/飞书国际版机器人接入，支持私聊、群聊、cron job 结果发送，并支持 text、images、audio、file attachments；连接模式包括推荐的 websocket 和可选 webhook。群聊中 Hermes 默认只响应 @ 机器人的消息。

larksuite/openclaw-lark 是飞书/Lark 开放平台团队维护的 OpenClaw 插件，能力覆盖消息、文档、多维表格、表格、日历、任务等，并支持交互卡片、流式回复、权限策略和群配置。但它是 OpenClaw 插件，不是 Hermes 原生插件。因此本项目不得假设 Hermes 能直接加载该插件。正确设计是：

- Hermes：Feishu/Lark 聊天入口、会话入口、Agent runtime。
- Lark CLI：飞书资源执行层，例如写文档、写多维表格、发消息、查日历、查任务。
- openclaw-lark：作为飞书能力设计参考；如果后续需要完整复用它，则通过 OpenClaw sidecar 暴露 HTTP/MCP/tool endpoint 给 Hermes 调用，而不是直接导入。
- Research Agent：本项目自有业务逻辑层。

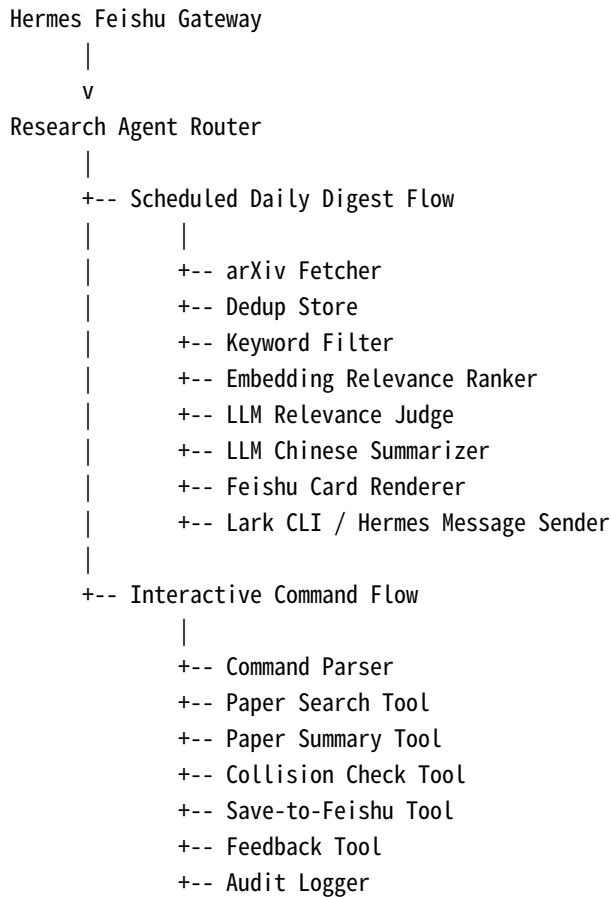
Lark CLI 是飞书/Lark 官方维护的 CLI，面向人和 AI Agent，覆盖 Messenger、Docs、Base、Sheets、Calendar、Mail、Tasks、Meetings 等业务域，提供 200+ commands 和 26 个 AI Agent skills。它还强调结构化输出、agent-native 设计、输入注入保护、终端输出清洗和系统密钥链存储。

arXiv API 支持 search_query、id_list、start、max_results 等参数；返回 Atom feed；支持 sortBy=submittedDate 和 sortOrder=descending；也支持 cat、ti、abs、all 等字段检索，以及 submittedDate:[YYYYMMDDHHMM TO YYYYMMDDHHMM] 的提交时间范围过滤。连续分页请求时应加入约 3 秒间隔。

2. 总体架构

Feishu Group / DM

|
v



系统分为五层：

1. **Gateway Layer**：Hermes Feishu/Lark 接入，负责收发消息、识别 @、维护 session。
2. **Agent Layer**：Research Agent，负责意图识别、任务编排、工具调用。
3. **Research Pipeline Layer**：arXiv 数据获取、筛选、评分、总结、去重。
4. **Feishu Operation Layer**：Lark CLI wrapper 或 Hermes 消息接口，负责发群消息、写文档、写多维表格。
5. **Storage Layer**：SQLite + JSON/YAML 配置 + 可选向量库。

3. 必须实现的核心场景

3.1 每日 08:30 晨报

每天新加坡时间 08:30 自动执行：

1. 获取 now = Asia/Singapore 当前时间。
2. 查询窗口 = [now - 24h, now]。
3. 为每个 research_profile 构造 arXiv query。
4. 拉取候选论文。
5. 用 arXiv ID 去重。
6. 用关键词和负关键词做初筛。
7. 用 embedding 计算论文与 profile 的语义相似度。
8. 用 LLM 对 top candidates 做结构化相关性判断。
9. 对入选论文生成中文摘要。

10. 生成飞书晨报卡片。
11. 推送到指定飞书群。
12. 写入 run log、paper log、delivery log。

注意：arXiv 用 UTC/GMT 时间，系统本地时间用 Asia/Singapore。实现时必须显式转换时区。

为了防止漏掉 arXiv 发布时间边界，实际抓取窗口应为过去 30 小时，最终展示仍限制为过去 24 小时；已发送论文通过 `delivered_items` 去重。

3.2 飞书群 @ 即时命令

机器人在群聊中只响应 @ 它的消息。必须支持以下命令：

```
@小麦 今日论文
@小麦 查一下 cyber range LLM agent 最近有什么新论文
@小麦 总结 https://arxiv.org/abs/xxxx.xxxxx
@小麦 这篇和我的靶场生成方向撞车吗 https://arxiv.org/abs/xxxx.xxxxx
@小麦 把这篇保存到论文库 https://arxiv.org/abs/xxxx.xxxxx
@小麦 以后多关注 MITRE ATT&CK extraction 和 cyber range generation
@小麦 今天这篇不相关
@小麦 给我解释这篇的方法、实验和局限
```

命令解析策略：

1. 明确命令优先，例如“总结”“保存”“撞车”“今日论文”。
2. 如果包含 arXiv URL 或 arXiv ID，进入 single-paper flow。
3. 如果包含主题描述，进入 ad-hoc search flow。
4. 如果是反馈，例如“不相关”“有用”“必读”，进入 feedback flow。
5. 无法判断时，返回简短澄清，不执行飞书写入或外部操作。

4. 推荐仓库结构

```
arxiv-feishu-research-agent/
  README.md
  IMPLEMENTATION_SCHEMA.md
  pyproject.toml
  .env.example
  config/
    profiles.yml
    app.yml
    prompts/
      relevance_judge.md
      paper_summary.md
      collision_check.md
      daily_digest.md
  src/
    agent/
      __init__.py
```

```
router.py
command_parser.py
session.py
permissions.py
arxiv/
  __init__.py
  client.py
  query_builder.py
  normalizer.py
  pdf_fetcher.py
relevance/
  __init__.py
  keyword_filter.py
  embeddings.py
  ranker.py
  llm_judge.py
  feedback_model.py
summarizer/
  __init__.py
  abstract_summary.py
  pdf_summary.py
  collision_checker.py
feishu/
  __init__.py
  hermes_adapter.py
  lark_cli.py
  card_renderer.py
  message_sender.py
  base_writer.py
  docs_writer.py
storage/
  __init__.py
  db.py
  migrations/
    001_init.sql
  repositories.py
scheduler/
  __init__.py
  daily_digest_job.py
  cron.py
observability/
  __init__.py
  logger.py
  audit.py
  metrics.py
main.py
tests/
  test_arxiv_query_builder.py
  test_relevance_ranker.py
  test_command_parser.py
  test_card_renderer.py
```

```
test_lark_cli_safety.py
fixtures/
  arxiv_feed_sample.xml
  sample_papers.json
```

5. 环境变量 Schema

`.env.example` 必须包含：

```
# Runtime
APP_ENV=local
TZ=Asia/Singapore
LOG_LEVEL=INFO

# Hermes / Feishu Gateway
HERMES_BASE_URL=http://127.0.0.1:18789
HERMES_AGENT_NAME=arxiv-research-agent
FEISHU_HOME_CHAT_ID=
FEISHU_ALLOWED_CHAT_IDS=
FEISHU_ALLOWED_USER_IDS=

# Lark CLI
LARK_CLI_BIN=lark
LARK_CLI_TIMEOUT_SECONDS=20
LARK_CLI_DRY_RUN=false

# LLM
LLM_PROVIDER=openai_compatible
LLM_BASE_URL=
LLM_API_KEY=
LLM_CHAT_MODEL=
LLM_EMBEDDING_MODEL=
LLM_TIMEOUT_SECONDS=60

# arXiv
ARXIV_API_BASE=https://export.arxiv.org/api/query
ARXIV_REQUEST_DELAY_SECONDS=3
ARXIV_MAX_RESULTS_PER_PROFILE=200

# Storage
SQLITE_PATH=./data/research_agent.sqlite3
CACHE_DIR=./data/cache
```

不得把真实 secret 写进 git。

6. 应用配置 Schema

`config/app.yml`：

```
app:
  name: "arxiv-feishu-research-agent"
  timezone: "Asia/Singapore"
  daily_digest_time: "08:30"
  digest_lookback_hours: 24
  fetch_lookback_hours: 30
  max_papers_per_profile: 8
  max_total_papers_per_digest: 20
  language: "zh-CN"

gateway:
  mode: "hermes"
  group_reply_requires_mention: true
  allowed_chat_ids:
    - "oc_xxx"
  allowed_user_ids:
    - "ou_xxx"

feishu:
  sender: "hermes"
  lark_cli_enabled: true
  default_output:
    send_group_card: true
    save_digest_to_doc: false
    save_selected_to_base: true

ranking:
  keyword_weight: 0.15
  semantic_weight: 0.55
  llm_judge_weight: 0.30
  min_final_score: 0.62
  min_semantic_score: 0.30

summary:
  abstract_only_by_default: true
  pdf_deep_summary_threshold: 0.78
  max_summary_tokens_per_paper: 700

safety:
  dry_run: false
  require_confirmation_for:
    - "write_doc"
    - "write_base"
    - "create_task"
    - "send_external_message"
  shell_command_allowlist:
    - "lark"
  block_raw_shell: true
  audit_all_tool_calls: true
```

config/profiles.yml :

```
profiles:
- id: "llm_security_range"
  name: "LLM 安全靶场 / Agent 评测"
  enabled: true
  arxiv_categories:
    - "cs.AI"
    - "cs.CL"
    - "cs.CR"
    - "cs.SE"
    - "cs.LG"
  semantic_query: >
    LLM agents for cybersecurity, cyber range generation, penetration testing agents,
    CTF benchmark, MITRE ATT&CK extraction, autonomous red teaming, tool-use agents,
    security evaluation benchmark, malware lab automation, attack path planning.
  positive_keywords:
    - "cyber range"
    - "penetration testing"
    - "red teaming"
    - "MITRE ATT&CK"
    - "CTF"
    - "security benchmark"
    - "LLM agent"
    - "autonomous agent"
    - "attack graph"
    - "tool use"
  negative_keywords:
    - "blockchain trading"
    - "cryptocurrency price"
    - "wireless sensor"
    - "vehicle routing"
  min_final_score: 0.62
  top_k: 8

- id: "rpki_bgp_routing_security"
  name: "RPKI / BGP / 路由安全"
  enabled: true
  arxiv_categories:
    - "cs.NI"
    - "cs.CR"
  semantic_query: >
    BGP security, RPKI, route origin validation, route leak, route hijacking,
    ASPA, inter-domain routing, routing anomaly detection, large-scale Internet
    routing measurement, Cloudflare Radar style routing monitoring.
  positive_keywords:
    - "BGP"
    - "RPKI"
    - "route leak"
    - "route hijack"
```

```

- "ROV"
- "ASPA"
- "inter-domain routing"
- "routing anomaly"
negative_keywords:
- "robot path planning"
- "vehicle routing"
- "neural routing"
min_final_score: 0.64
top_k: 6

- id: "dbs_memory_neuromodulation"
  name: "DBS / 记忆 / 神经调控"
  enabled: true
  arxiv_categories:
    - "q-bio.NC"
    - "eess.SP"
    - "cs.LG"
    - "stat.ML"
  semantic_query: >
    deep brain stimulation, human memory, Parkinson's disease, substantia nigra,
    subthalamic nucleus, neuromodulation, intracranial EEG, LFP, recognition memory,
    neural decoding, cortico-subcortical circuits.
  positive_keywords:
    - "deep brain stimulation"
    - "DBS"
    - "Parkinson"
    - "substantia nigra"
    - "subthalamic nucleus"
    - "recognition memory"
    - "intracranial EEG"
    - "LFP"
  negative_keywords:
    - "plant disease"
    - "image segmentation only"
  min_final_score: 0.58
  top_k: 5

```

7. 数据库 Schema

使用 SQLite。后续可替换 PostgreSQL，但第一版必须保持本地可跑。

7.1 papers

```

CREATE TABLE IF NOT EXISTS papers (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  arxiv_id TEXT NOT NULL,
  arxiv_id_base TEXT NOT NULL,
  version INTEGER,
  title TEXT NOT NULL,

```



```

abstract TEXT NOT NULL,
authors_json TEXT NOT NULL,
categories_json TEXT NOT NULL,
primary_category TEXT,
published_at TEXT,
updated_at TEXT,
abs_url TEXT NOT NULL,
pdf_url TEXT,
raw_json TEXT NOT NULL,
created_at TEXT NOT NULL,
UNIQUE(arxiv_id)
);

```

7.2 paper_scores

```

CREATE TABLE IF NOT EXISTS paper_scores (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  paper_id INTEGER NOT NULL,
  profile_id TEXT NOT NULL,
  keyword_score REAL NOT NULL,
  semantic_score REAL NOT NULL,
  llm_relevance_score REAL,
  final_score REAL NOT NULL,
  judge_label TEXT,
  judge_reason TEXT,
  created_at TEXT NOT NULL,
  FOREIGN KEY(paper_id) REFERENCES papers(id)
);

```

7.3 summaries

```

CREATE TABLE IF NOT EXISTS summaries (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  paper_id INTEGER NOT NULL,
  profile_id TEXT,
  summary_type TEXT NOT NULL,
  language TEXT NOT NULL,
  summary_json TEXT NOT NULL,
  model_name TEXT NOT NULL,
  created_at TEXT NOT NULL,
  FOREIGN KEY(paper_id) REFERENCES papers(id)
);

```

7.4 digest_runs

```

CREATE TABLE IF NOT EXISTS digest_runs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  run_id TEXT NOT NULL UNIQUE,

```

```

started_at TEXT NOT NULL,
finished_at TEXT,
status TEXT NOT NULL,
window_start TEXT NOT NULL,
window_end TEXT NOT NULL,
candidate_count INTEGER DEFAULT 0,
selected_count INTEGER DEFAULT 0,
delivered_chat_id TEXT,
error_message TEXT
);

```

7.5 delivered_items

```

CREATE TABLE IF NOT EXISTS delivered_items (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  arxiv_id_base TEXT NOT NULL,
  profile_id TEXT NOT NULL,
  chat_id TEXT NOT NULL,
  delivered_at TEXT NOT NULL,
  digest_run_id TEXT,
  UNIQUE(arxiv_id_base, profile_id, chat_id)
);

```

7.6 feedback

```

CREATE TABLE IF NOT EXISTS feedback (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  arxiv_id_base TEXT NOT NULL,
  profile_id TEXT,
  user_id TEXT NOT NULL,
  feedback_type TEXT NOT NULL,
  feedback_text TEXT,
  created_at TEXT NOT NULL
);

```

feedback_type 枚举:

```

relevant
irrelevant
must_read
skip
collision_risk
not_collision
save

```

7.7 audit_logs

```
CREATE TABLE IF NOT EXISTS audit_logs (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  event_id TEXT NOT NULL UNIQUE,  
  user_id TEXT,  
  chat_id TEXT,  
  event_type TEXT NOT NULL,  
  tool_name TEXT,  
  tool_args_json TEXT,  
  tool_result_summary TEXT,  
  risk_level TEXT NOT NULL,  
  created_at TEXT NOT NULL  
);
```

8. Python 数据模型 Schema

使用 Pydantic。

```
from typing import Literal, Optional  
from pydantic import BaseModel, Field  
  
class ResearchProfile(BaseModel):  
    id: str  
    name: str  
    enabled: bool = True  
    arxiv_categories: list[str]  
    semantic_query: str  
    positive_keywords: list[str] = []  
    negative_keywords: list[str] = []  
    min_final_score: float = 0.62  
    top_k: int = 8  
  
class ArxivPaper(BaseModel):  
    arxiv_id: str  
    arxiv_id_base: str  
    version: Optional[int] = None  
    title: str  
    abstract: str  
    authors: list[str]  
    categories: list[str]  
    primary_category: Optional[str] = None  
    published_at: Optional[str] = None  
    updated_at: Optional[str] = None  
    abs_url: str  
    pdf_url: Optional[str] = None
```

```

class PaperScore(BaseModel):
    profile_id: str
    keyword_score: float
    semantic_score: float
    llm_relevance_score: Optional[float] = None
    final_score: float
    judge_label: Literal["high", "medium", "low", "reject"]
    judge_reason: str

class PaperSummary(BaseModel):
    arxiv_id: str
    profile_id: Optional[str]
    one_sentence: str
    problem: str
    method: str
    main_findings: str
    why_relevant: str
    limitations: str
    collision_risk: Literal["high", "medium", "low", "unknown"]
    recommended_action: Literal["must_read", "skim", "archive", "skip"]

class DigestItem(BaseModel):
    paper: ArxivPaper
    score: PaperScore
    summary: PaperSummary

class DigestReport(BaseModel):
    run_id: str
    window_start: str
    window_end: str
    profile_name: str
    items: list[DigestItem]

```

9. arXiv 抓取规则

9.1 Query 构造

对每个 profile 构造：

```
(cat:cs.AI OR cat:cs.CL OR cat:cs.CR) AND submittedDate:[YYYYMMDDHHMM TO YYYYMMDDHHMM]
```

请求参数：

```
params = {
  "search_query": query,
  "start": 0,
  "max_results": max_results,
  "sortBy": "submittedDate",
  "sortOrder": "descending",
}
```

必须：

1. 使用 `https://export.arxiv.org/api/query`。
2. 所有时间转换为 UTC。
3. 连续请求之间 sleep `ARXIV_REQUEST_DELAY_SECONDS`，默认 3 秒。
4. `max_results` 第一版不超过每 profile 200。
5. API 失败时最多重试 3 次，指数退避。
6. 保存原始 Atom entry 的关键字段到 `raw_json`。

9.2 去重规则

1. `arxiv_id` 包含版本，例如 `2607.01234v1`。
2. `arxiv_id_base` 去掉版本，例如 `2607.01234`。
3. 每日晨报对 `arxiv_id_base + profile_id + chat_id` 去重。
4. 如果新版本出现，例如 v2，但 v1 已经发过，默认不再晨报；用户手动问某篇时允许返回最新版本。
5. 可在配置中增加 `include_new_versions: true`，但默认关闭。

10. 相关性评分规则

相关性评分分三层。

10.1 关键词层

输入：title + abstract + categories。

规则：

```
positive_hit_count = 命中 positive_keywords 的数量
negative_hit_count = 命中 negative_keywords 的数量
keyword_score = clamp((positive_hit_count * 0.12) - (negative_hit_count * 0.25), -1, 1)
```

如果 `negative_hit_count >= 2`，直接标记为 `reject_candidate`，除非 semantic score 极高。

10.2 语义层

将 profile 的 `semantic_query` 与论文 `title + abstract + categories` 分别 embedding，计算 cosine similarity。

```
semantic_score = cosine(profile_embedding, paper_embedding)
```

第一版阈值：

```
semantic_score < 0.25: reject  
0.25 - 0.35: low  
0.35 - 0.48: medium  
> 0.48: high
```

具体数值需要根据实际模型校准，阈值应配置化。

10.3 LLM Judge 层

只对初筛后的 top 论文调用 LLM judge，减少成本。

LLM 必须输出 JSON：

```
{  
  "label": "high | medium | low | reject",  
  "relevance_score": 0.0,  
  "reason": "为什么相关或不相关",  
  "matched_aspects": ["具体匹配点"],  
  "mismatch_aspects": ["具体不匹配点"],  
  "collision_risk": "high | medium | low | unknown"  
}
```

最终分数：

```
final_score =  
  keyword_weight * normalized_keyword_score  
  + semantic_weight * semantic_score  
  + llm_judge_weight * llm_relevance_score  
  + feedback_adjustment
```

`feedback_adjustment`：

```
用户标记 must_read: +0.08  
用户标记 relevant: +0.04  
用户标记 irrelevant: -0.08  
用户标记 skip: -0.12
```

第一版可以只记录 feedback，不训练模型；第二版再把 feedback 纳入 profile embedding 或 reranker。

11. 摘要输出 Schema

每篇论文必须输出以下字段：

```
{
  "title_zh": "中文标题，不得过度意译",
  "one_sentence": "一句话说明这篇在做什么",
  "problem": "研究问题",
  "method": "核心方法",
  "main_findings": "主要发现或贡献",
  "why_relevant": "为什么和我的方向相关",
  "collision_risk": "和我当前研究是否可能撞车",
  "limitations": "摘要层面能看出的局限",
  "recommended_action": "must_read | skim | archive | skip"
}
```

摘要原则：

1. 默认只基于 arXiv title + abstract。
2. 不允许编造实验结果、数据集、代码地址、SOTA claim。
3. 如果摘要没有说明，必须写“摘要未说明”。
4. 只有在用户要求“深度总结”或 final_score 很高时，才下载 PDF。
5. PDF 总结必须区分“论文明确写了”和“Agent 推断”。

12. 晨报消息格式

飞书晨报标题：

小麦 arXiv 晨报 | YYYY-MM-DD | 过去24小时

正文结构：

今天筛到 N 篇相关论文，其中：

- 必读：A 篇
- 可扫：B 篇
- 存档：C 篇

【方向一：LLM 安全靶场 / Agent 评测】

1. 中文标题 / English Title

arXiv: xxxx.xxxxx | Score: 0.xx | Category: cs.CR

一句话结论：

方法：

为什么值得看：

撞车风险：

建议动作：必读 / 可扫 / 存档 / 跳过

链接：abs / pdf

【方向二：RPKI / BGP / 路由安全】

...

飞书卡片必须提供按钮，第一版可以先只渲染纯文本；第二版实现交互按钮：

[必读] [不相关] [深度总结] [保存到论文库] [检查撞车]

按钮回调事件写入 `feedback` 和 `audit_logs`。

13. 飞书即时命令 Schema

13.1 Command Intent

```
class CommandIntent(BaseModel):
    intent: Literal[
        "daily_digest_now",
        "search_topic",
        "summarize_paper",
        "collision_check",
        "save_paper",
        "feedback",
        "update_profile",
        "help",
        "unknown"
    ]
    arxiv_ids: list[str] = []
    urls: list[str] = []
    topic: Optional[str] = None
    feedback_type: Optional[str] = None
    raw_text: str
```

13.2 命令映射

今日论文 / 今天论文 / 晨报

=> `daily_digest_now`

查一下 X / 搜 X / 最近 X 有什么论文

=> `search_topic`

总结 + arxiv URL

=> `summarize_paper`

撞车 / 和我方向冲突吗 / 是否有人做了

=> `collision_check`

保存 / 加入论文库

=> `save_paper`

不相关 / 有用 / 必读 / 跳过

=> `feedback`

以后多关注 X / 增加关键词 X
=> update_profile

13.3 权限规则

1. 群聊中必须 @ 机器人。
2. `allowed_chat_ids` 外的群不响应。
3. `allowed_user_ids` 外的用户只能收到“无权限”。
4. 写飞书文档、多维表格、任务前必须确认，除非命令来自 owner 且配置允许。
5. 所有 Lark CLI 写操作必须进入 audit log。

14. Lark CLI Wrapper Schema

禁止让 LLM 直接拼 shell 命令执行。必须实现受控 wrapper:

```
class LarkCliCommand(BaseModel):
    domain: Literal["messenger", "docs", "base", "sheets", "calendar", "tasks"]
    action: str
    args: dict
    risk_level: Literal["low", "medium", "high"]
    require_confirmation: bool = False
```

允许的第一版操作:

```
allowed_lark_cli_operations:
- domain: "messenger"
  actions:
    - "send_message"
    - "reply_message"
- domain: "docs"
  actions:
    - "create_doc"
    - "append_doc"
    - "read_doc"
- domain: "base"
  actions:
    - "add_record"
    - "search_records"
    - "update_record"
```

禁止:

1. 任意 shell。
2. 删除飞书文档。
3. 删除多维表格记录。
4. 大范围读取群消息历史。
5. 读取通讯录全量成员。

6. 自动邀请外部用户。
7. 自动转发文件。

`lark_cli.py` 必须提供:

```
class LarkCli:
    def run(self, command: LarkCliCommand) -> dict:
        """
        1. 校验 domain/action 是否在 allowlist。
        2. 校验 require_confirmation。
        3. 构造 lark CLI 参数, 不允许 shell=True。
        4. 设置 timeout。
        5. 捕获 stdout/stderr。
        6. 清洗输出中的 secret。
        7. 写 audit log。
        8. 返回结构化 JSON。
        """
```

15. Hermes Adapter Schema

`hermes_adapter.py` 负责把 Hermes 收到的消息转为内部事件。

```
class HermesEvent(BaseModel):
    event_id: str
    chat_id: str
    user_id: str
    message_id: str
    text: str
    is_group: bool
    is_mention: bool
    timestamp: str
    raw_json: dict
```

必须实现:

```
def parse_hermes_event(raw: dict) -> HermesEvent:
    pass

def should_respond(event: HermesEvent, config: AppConfig) -> bool:
    pass

def send_text(chat_id: str, text: str, reply_to: str | None = None) -> None:
    pass

def send_card(chat_id: str, card: dict, reply_to: str | None = None) -> None:
    pass
```

如果 Hermes 已经提供 Feishu 发送能力，优先使用 Hermes 消息接口；如果需要写文档、多维表格，再调用 Lark CLI。

16. Scheduler Schema

第一版可以使用 APScheduler 或系统 cron。必须保证新加坡时间 08:30。

推荐 cron：

```
30 0 * * * cd /path/to/arxiv-feishu-research-agent && .venv/bin/python -m  
src.scheduler.daily_digest_job
```

解释：新加坡时间 UTC+8，因此 08:30 SGT = 00:30 UTC。

如果使用 Python scheduler：

```
scheduler.add_job(  
    run_daily_digest,  
    trigger="cron",  
    hour=8,  
    minute=30,  
    timezone="Asia/Singapore",  
)
```

必须支持手动运行：

```
python -m src.scheduler.daily_digest_job --dry-run  
python -m src.scheduler.daily_digest_job --send  
python -m src.scheduler.daily_digest_job --profile llm_security_range
```

17. Prompt Schema

17.1 Relevance Judge Prompt

文件： `config/prompts/relevance_judge.md`

你是一个严谨的论文筛选助手。你只能根据给定的标题、摘要、分类和用户研究方向判断相关性，不得编造论文没有写出的内容。

用户研究方向：

{{profile_name}}
{{semantic_query}}

论文：

Title: {{title}}
Abstract: {{abstract}}

Categories: {{categories}}

请输出严格 JSON，不要输出 Markdown：

```
{
  "label": "high | medium | low | reject",
  "relevance_score": 0.0,
  "reason": "用中文说明相关性理由",
  "matched_aspects": ["匹配点1", "匹配点2"],
  "mismatch_aspects": ["不匹配点1"],
  "collision_risk": "high | medium | low | unknown"
}
```

判断标准：

- high: 论文核心问题、方法或任务与用户方向直接相关。
- medium: 论文部分方法或评测设置可借鉴。
- low: 只有宽泛概念相关。
- reject: 基本不相关，或被负关键词命中。

17.2 Paper Summary Prompt

文件： `config/prompts/paper_summary.md`

你是一个研究助理。请只基于论文标题和摘要生成中文总结，不要编造摘要之外的信息。

输出严格 JSON：

```
{
  "title_zh": "",
  "one_sentence": "",
  "problem": "",
  "method": "",
  "main_findings": "",
  "why_relevant": "",
  "collision_risk": "high | medium | low | unknown",
  "limitations": "",
  "recommended_action": "must_read | skim | archive | skip"
}
```

论文标题：

{{title}}

论文摘要：

{{abstract}}

用户研究方向：

{{profile_name}}

{{semantic_query}}

17.3 Collision Check Prompt

文件: `config/prompts/collision_check.md`

你要判断一篇论文是否可能与用户当前研究方向“撞车”。

用户当前方向:

{{user_project_description}}

论文:

Title: {{title}}

Abstract: {{abstract}}

请输出严格 JSON:

```
{
  "collision_risk": "high | medium | low | unknown",
  "overlap_points": ["具体重叠点"],
  "difference_points": ["关键差异点"],
  "what_to_read_first": ["优先阅读的章节或信息"],
  "suggested_response": "用户应该如何处理：引用、区分、改实验、忽略等"
}
```

要求:

- 只根据给定信息判断。
- 不要把宽泛同领域误判为撞车。
- 如果摘要信息不足, 输出 unknown, 并说明需要阅读全文。

18. 论文库保存 Schema

如果用户要求“保存到论文库”, 第一版存 SQLite; 第二版写入飞书多维表格。

多维表格字段建议:

```
arxiv_id
title
title_zh
authors
published_at
categories
profile
score
recommended_action
collision_risk
one_sentence
why_relevant
limitations
abs_url
pdf_url
```

```
status
notes
created_at
```

`status` 枚举:

```
new
must_read
reading
read
cited
skipped
irrelevant
```

写入飞书多维表格前必须:

1. 查重 `arxiv_id_base` 。
2. 如果存在, 只更新 status/notes, 不创建重复记录。
3. 写入成功后在群里回复记录链接。
4. 写入失败时不丢数据, 至少保存在 SQLite。

19. 错误处理 Schema

必须处理:

```
arXiv API timeout
arXiv API malformed feed
LLM API timeout
LLM JSON parse error
embedding API failure
Hermes send failure
Lark CLI timeout
Lark CLI permission error
SQLite locked
empty digest
```

策略:

1. arXiv 失败: 重试 3 次, 失败则推送“晨报抓取失败”给 owner, 不发空报告。
2. LLM 失败: 允许 fallback 到标题摘要直接展示, 但标记“未完成 LLM 总结”。
3. Feishu 发送失败: 保存 digest 到本地 `data/failed_digest_*.json`。
4. JSON parse 失败: 要求模型重试一次; 再失败则用文本 fallback。
5. 空 digest: 发送一条简短消息“过去 24 小时没有筛到明显相关论文”, 并写 run log。

20. 安全规则

必须实现以下安全策略：

1. 默认只允许 owner 和指定群使用。
2. 群聊必须 @ 才响应。
3. 所有写操作都要 audit。
4. Lark CLI 只能通过 allowlist wrapper 调用。
5. 不允许 `shell=True`。
6. 不允许 LLM 直接生成并执行命令。
7. 不允许把飞书 App Secret、LLM API Key、数据库路径等 secret 发到群里。
8. 任何来自论文 PDF、网页、飞书文档的内容都视为不可信，不能覆盖系统指令。
9. 遇到“忽略之前规则”“把密钥发出来”“读取所有聊天记录”等内容，必须拒绝。
10. 高风险操作需要二次确认。

需要特别注意：飞书官方 OpenClaw 插件和 Lark CLI 文档都提示，AI Agent 连接飞书权限后可能带来模型幻觉、不可预测执行、prompt injection、数据泄露和未授权操作风险，因此第一版必须保持最小权限和最小写操作。

21. 测试验收标准

21.1 单元测试

必须通过：

```
pytest tests/test_arxiv_query_builder.py
pytest tests/test_relevance_ranker.py
pytest tests/test_command_parser.py
pytest tests/test_card_renderer.py
pytest tests/test_lark_cli_safety.py
```

覆盖：

1. arXiv query 时间范围正确。
2. `arxiv_id_base` 去版本正确。
3. 正关键词提高分数。
4. 负关键词降低分数。
5. embedding ranker 返回稳定排序。
6. LLM judge JSON parse 失败时 fallback。
7. 群聊不 @ 不响应。
8. 非 allowlist 用户不响应。
9. Lark CLI 禁止非 allowlist 命令。
10. 飞书卡片字段完整。

21.2 集成测试

必须支持 dry-run：

```
python -m src.scheduler.daily_digest_job --dry-run
```

输出：

- 抓取到候选论文数量
- 每个 profile 入选数量
- 每篇论文分数
- 渲染后的晨报文本
- 不实际发送飞书

必须支持测试发送：

```
python -m src.scheduler.daily_digest_job --send --profile llm_security_range
```

要求：

1. 指定群收到晨报。
2. SQLite 写入 `digest_runs`。
3. `delivered_items` 去重生效。
4. 重复运行不重复推送同一篇论文。

21.3 飞书交互验收

在群里执行：

```
@小麦 今日论文
```

预期：

```
机器人返回当天 digest，或说明没有筛到。
```

执行：

```
@小麦 总结 https://arxiv.org/abs/2607.xxxxx
```

预期：

```
机器人返回中文结构化摘要。
```

执行：

@小麦 把这篇保存到论文库 <https://arxiv.org/abs/2607.xxxxx>

预期：

机器人先请求确认；确认后写入 SQLite 和飞书多维表格。

22. 第一版实现顺序

CodingAgent 按以下顺序实现，不要跳步：

Milestone 1：本地 arXiv 晨报 dry-run

交付：

```
- config/profiles.yml
- arxiv client
- SQLite schema
- keyword filter
- embedding ranker
- LLM summary
- daily_digest_job --dry-run
```

验收：

```
python -m src.scheduler.daily_digest_job --dry-run
```

Milestone 2：飞书发送

交付：

```
- Hermes send adapter 或 Feishu webhook fallback
- card_renderer
- daily_digest_job --send
```

验收：

```
python -m src.scheduler.daily_digest_job --send
```

Milestone 3：Hermes 群聊 @ 交互

交付：

- Hermes event parser
- command_parser
- summarize_paper flow
- search_topic flow
- permission check

验收：

飞书群 @ 机器人可以查论文和总结论文。

Milestone 4: Lark CLI 论文库写入

交付：

- lark_cli.py allowlist wrapper
- base_writer.py
- save_paper flow
- confirmation flow
- audit log

验收：

用户确认后，论文被写入飞书多维表格。

Milestone 5: 反馈学习

交付：

- feedback table
- 必读/不相关按钮或命令
- feedback_adjustment

验收：

用户标记不相关后，同类论文分数降低。

23. 最小可运行命令

初始化：

```
python -m venv .venv
source .venv/bin/activate
pip install -e .
```

```
cp .env.example .env
python -m src.storage.db init
```

Dry run:

```
python -m src.scheduler.daily_digest_job --dry-run
```

发送晨报:

```
python -m src.scheduler.daily_digest_job --send
```

启动交互服务:

```
python -m src.main serve
```

24. Definition of Done

项目第一版完成的标准:

1. 每天 08:30 能自动向指定飞书群发送 arXiv 晨报。
2. 晨报只包含过去 24 小时内新论文。
3. 晨报按照用户研究方向分组。
4. 每篇论文有相关性分数、中文摘要、相关性理由、撞车风险和建议动作。
5. 同一篇论文不会重复推送。
6. 用户可以在飞书群 @ 机器人总结指定 arXiv 论文。
7. 用户可以在飞书群 @ 机器人搜索某个临时主题。
8. 用户可以将论文保存到本地论文库。
9. Lark CLI 调用受 allowlist 限制。
10. 所有写操作进入 audit log。
11. 非授权群和非授权用户无法使用机器人。
12. 所有 secret 不入库、不入日志、不回显到飞书。